COMPUTER NETWORK

LAB WORKSHEET 3

AMITH GEO JOSE

202217B2017

Tasks need to perform on OSHA-LAB Virtual Machine	
Experiment 1:	Socket programming using any one of the programming languages(2M)

	Server Code (socket_server.c):
	
	#include <stdio.h>
	#include <stdlib.h>
	#include <string.h>
	#include <unistd.h>
	#include <arpa/inet.h>
	
	int main() {
	  char * ip = "127.0.0.1";
	  int port = 5566;
	  int server_sock, client_sock;
	  struct sockaddr_in server_addr, client_addr;
	  socklen_t addr_size;
	  char buffer[1024];
	  int n;
	
	  server_sock = socket(AF_INET, SOCK_STREAM, 0);
	  if (server_sock < 0) {
	  perror("Error in Creating Socket.....");
	  exit(1);
	  }
	  printf("TCP server socket created.\n");
	
	  memset(&server_addr, '\0', sizeof(server_addr));
	  server_addr.sin_family = AF_INET;
	  server_addr.sin_port = port;
	  server_addr.sin_addr.s_addr = inet_addr(ip);
	
	  n = bind(server_sock, (struct sockaddr*)&server_addr, sizeof(server_addr));
	  if (n < 0) {
	  perror("Bind error");
	  exit(1);
	  }

	printf("Bind to the port number: %d\n", port);
	
	listen(server_sock, 5);
	printf("Listening...\n");
	
	while (1) {
	addr_size = sizeof(client_addr);
	client_sock = accept(server_sock, (struct sockaddr*)&client_addr, &addr_size);
	printf("Client connected.\n");
	
	bzero(buffer, 1024);
	recv(client_sock, buffer, sizeof(buffer), 0);
	printf("Client: %s\n", buffer);
	
	bzero(buffer, 1024);
	strcpy(buffer, "HI, THIS IS SERVER. My BITS ID is <Your_Bits_ID>");
	printf("Server: %s\n", buffer);
	send(client_sock, buffer, strlen(buffer), 0);
	
	close(client_sock);
	printf("Client disconnected.\n\n");
	}
	return 0;
	}

Terminal 1, run	$ gcc socket_server.c -o server $ ./server

	Client Code (socket_client.c):
	
	#include <arpa/inet.h>
	#include <stdio.h>
	#include <stdlib.h>
	#include <string.h>
	#include <unistd.h>
	
	int main() {
	char *ip = "127.0.0.1";
	int port = 5566;
	int sock;
	struct sockaddr_in addr;
	socklen_t addr_size;
	char buffer[1024];
	int n;
	
	sock = socket(AF_INET, SOCK_STREAM, 0);

	if (sock < 0) {
	perror("Error in Creating.....");
	exit(1);
	}
	printf("TCP server socket created.\n");
	
	memset(&addr, '\0', sizeof(addr));
	addr.sin_family = AF_INET;
	addr.sin_port = port;
	addr.sin_addr.s_addr = inet_addr(ip);
	
	connect(sock, (struct sockaddr *)&addr, sizeof(addr));
	printf("Connected to the server.\n");
	
	bzero(buffer, 1024);
	strcpy(buffer, "HELLO, THIS IS CLIENT...What is your BITS ID.");
	printf("Client: %s\n", buffer);
	send(sock, buffer, strlen(buffer), 0);
	
	bzero(buffer, 1024);
	recv(sock, buffer, sizeof(buffer), 0);
	printf("Server: %s\n", buffer);
	
	close(sock);
	printf("Disconnected from the server.\n");
	
	return 0;
	}

Terminal 2, run:	$ gcc socket_client.c -o client $./client
	Open Two terminals Run **socket_server.c** program in One terminal and **socket_client.c** in other terminal. (Note: Server program has to start first and then run the client program)
	Paste the screen shots of the output for the experiment 1

Experiment 2	**Building different types of networks for experimenting with different types of applications using Network Simulators(ns-3 Tool)(2M)**
Steps to follow for reference if students can find better approach they can use	Step 1: Open Terminal Step 2: Go to NS-3 Directory **cd ns-allinone-3.43/ns-3.**43 Step 3: Create Program File **nano scratch/exp4-udp.cc** Step 4: Write the Program Step 5: Save and Exit **Press:** • **CTRL + O → Enter** • **CTRL + X** Step 6: Build NS-3 **./ns3 build** Step 7: Run the Program **./ns3 run scratch/exp4-udp** Step 8: Observe Output UDP packets are sent from client node to server node, confirming successful execution.
Code :	#include "ns3/core-module.h" #include "ns3/network-module.h" #include "ns3/internet-module.h" #include "ns3/point-to-point-module.h" #include "ns3/applications-module.h" using namespace ns3; NS_LOG_COMPONENT_DEFINE ("Experiment4UDP"); int main (int argc, char *argv[]) {

```
// Enable logging
LogComponentEnable ("Experiment4UDP", LOG_LEVEL_INFO);
LogComponentEnable ("UdpClient", LOG_LEVEL_INFO);
LogComponentEnable ("UdpServer", LOG_LEVEL_INFO);

// Step 1: Create nodes
NodeContainer nodes;
nodes.Create (2);

// Step 2: Configure point-to-point link
PointToPointHelper p2p;
p2p.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
p2p.SetChannelAttribute ("Delay", StringValue ("2ms"));

// Step 3: Install devices
NetDeviceContainer devices = p2p.Install (nodes);

// Step 4: Install Internet stack
InternetStackHelper stack;
stack.Install (nodes);

// Step 5: Assign IP addresses
Ipv4AddressHelper address;
address.SetBase ("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer interfaces = address.Assign (devices);

// Step 6: Create UDP Server (Node 1)
uint16_t port = 4000;
UdpServerHelper server (port);
ApplicationContainer serverApp = server.Install (nodes.Get (1));
serverApp.Start (Seconds (1.0));
serverApp.Stop (Seconds (10.0));
```

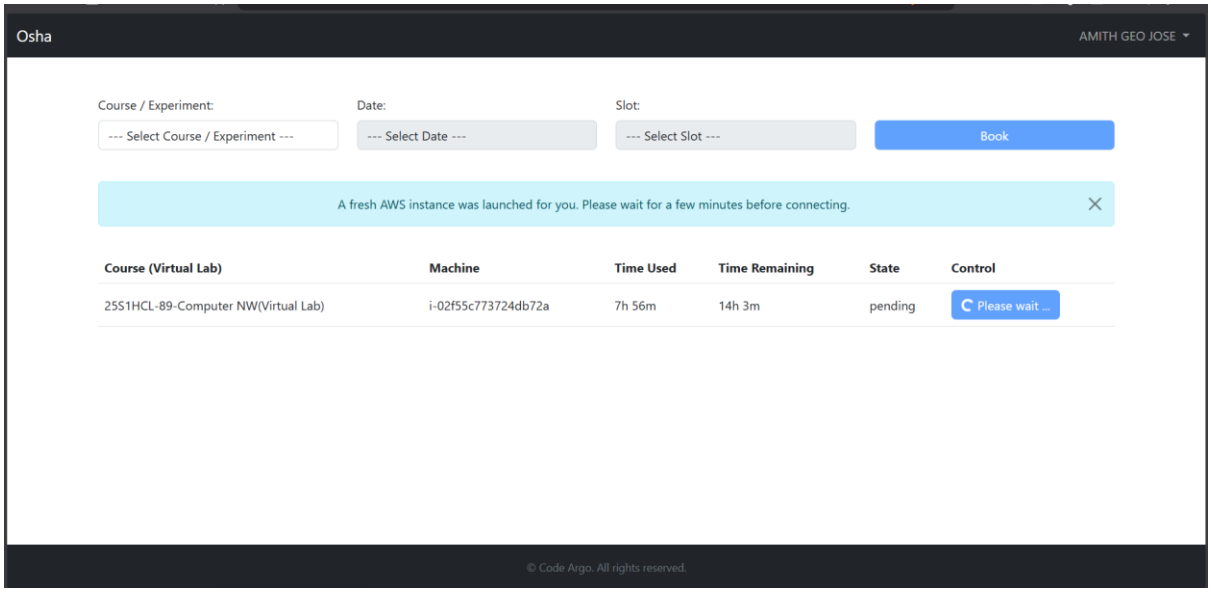
	// Step 7: Create UDP Client (Node 0)
	UdpClientHelper client (interfaces.GetAddress (1), port);
	client.SetAttribute ("MaxPackets", UintegerValue (5));
	client.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
	client.SetAttribute ("PacketSize", UintegerValue (1024));
	
	ApplicationContainer clientApp = client.Install (nodes.Get (0));
	clientApp.Start (Seconds (2.0));
	clientApp.Stop (Seconds (10.0));
	
	// Step 8: Run simulation
	Simulator::Run ();
	Simulator::Destroy ();
	return 0;
	}

Output	Paste the screen shots of these below for Experiment 2 · UDP client sends packets · UDP server receives packets · Packet transmission is displayed in terminal
Experiment 3	**Explain different examples provided in the labware modules in the eLearn course page.**
Students to prepare	**Design a simple network using NS-3 and simulate TCP-based client–server communication to study reliable data transmission** Students have to prepare code and paste the code and screen shot for this experiment 3 with the reference experience of experiment 1 n 2

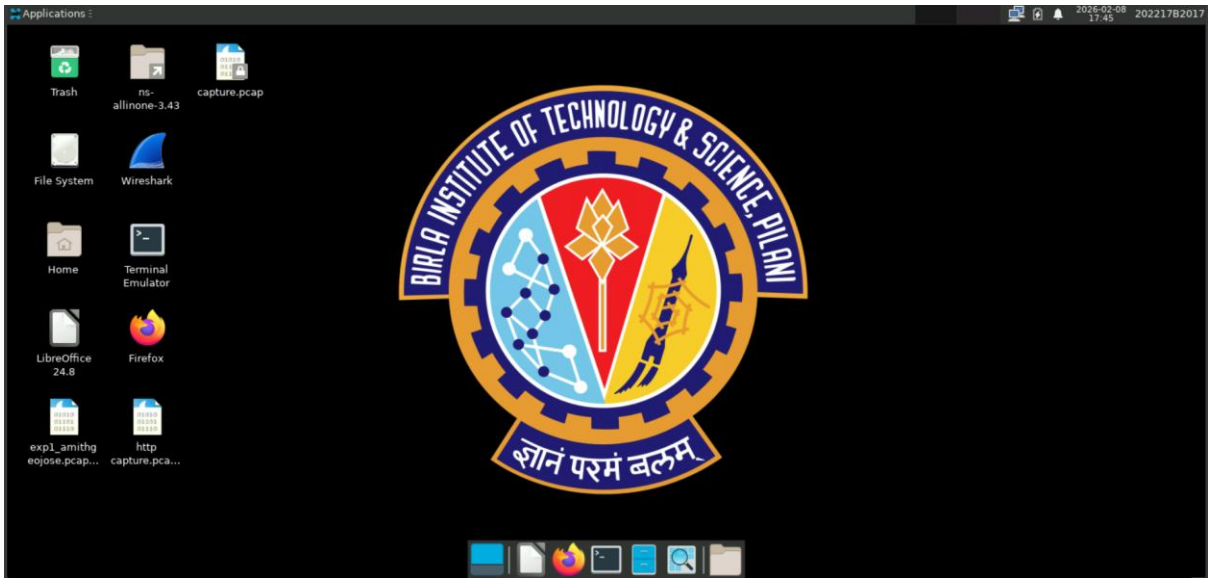
ANSWERS:

Sl. No	Name as appeared in Taxila	BITS ID No.
1.	AMITH GEO JOSE	202217B2017

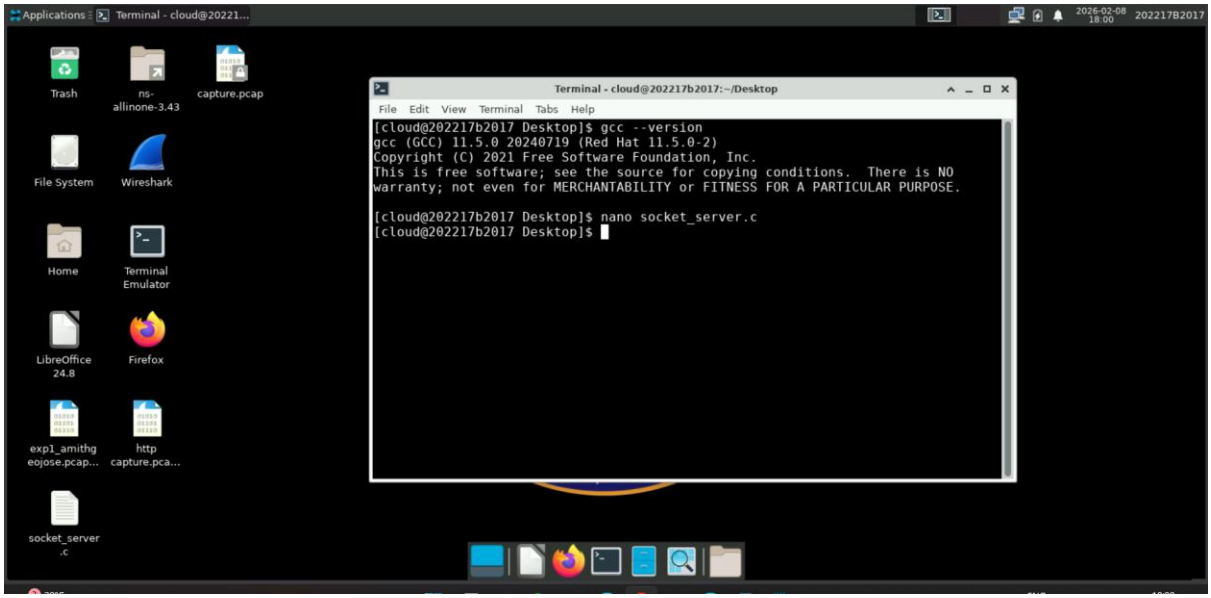
Screenshot of Lab before starting experiments:

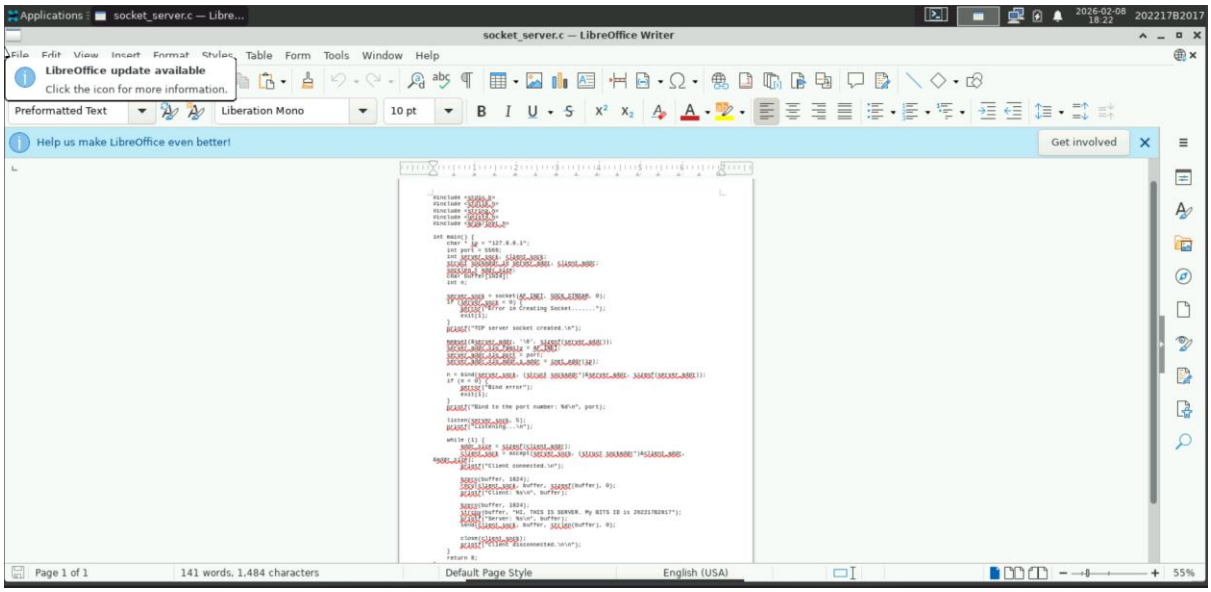


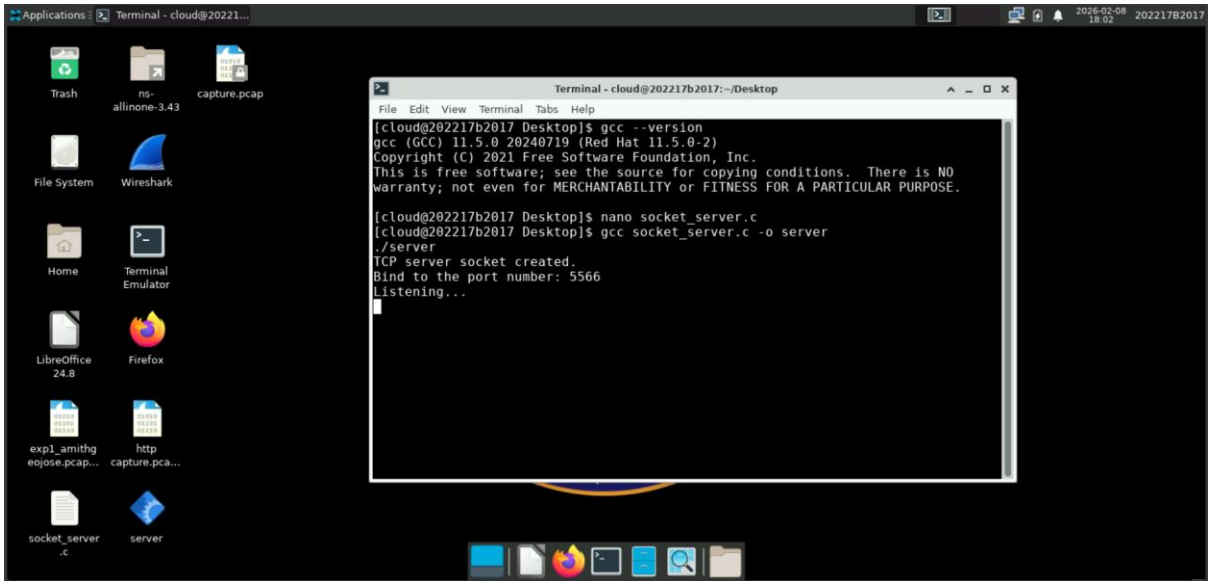
Task 1 : **Change Username**

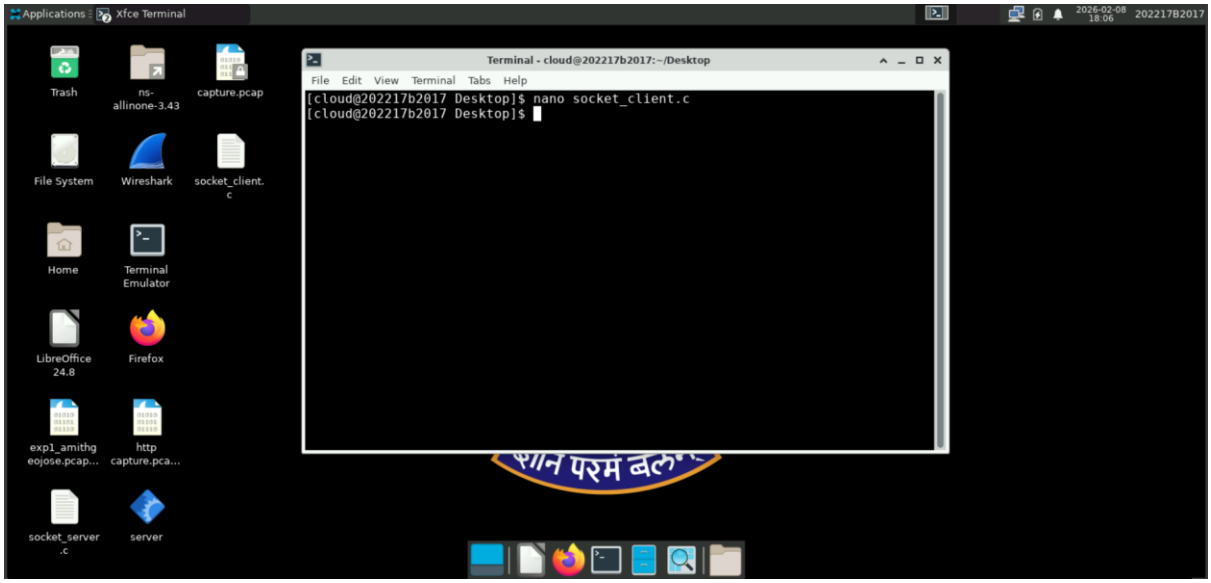


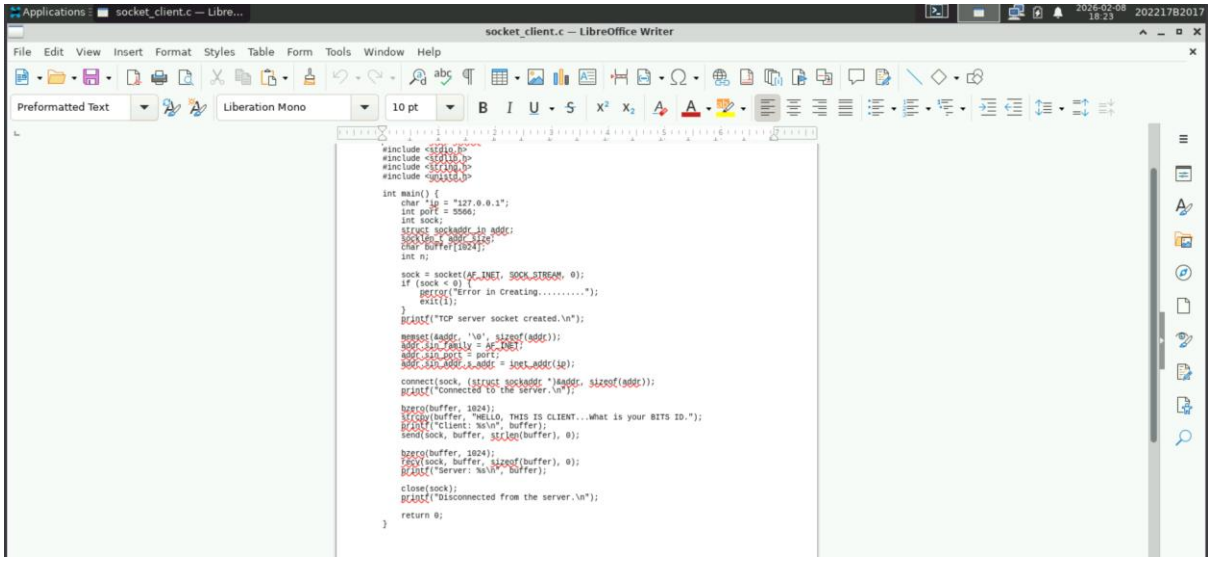
- Experiment 1: **Socket programming using any one of the programming languages**

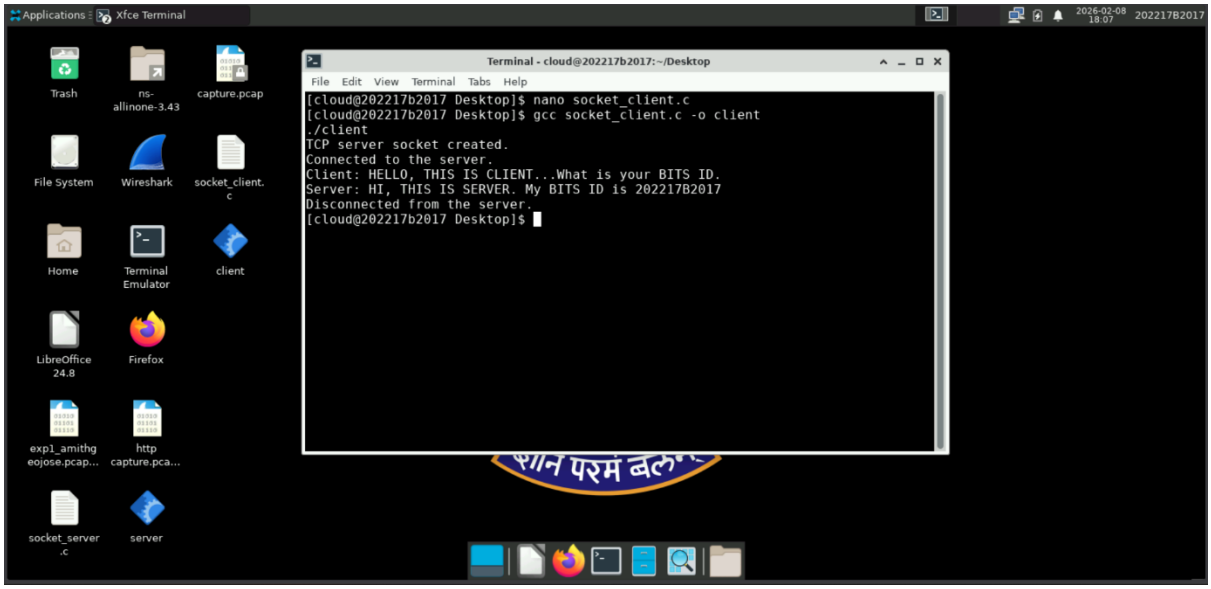





Applications
Terminal - cloud@20221...
2026-02-08 18:02
202217B2017
Trash
ns-allinone-3.43
capture.pcap
File System
Wireshark
Home
Terminal Emulator
LibreOffice 24.8
Firefox
exp1_amithgeojose.pcap...
http capture.pca...
socket_server.c
server
Terminal - cloud@202217b2017:~/Desktop
File Edit View Terminal Tabs Help
[cloud@202217b2017 Desktop]$ gcc --version
gcc (GCC) 11.5.0 20240719 (Red Hat 11.5.0-2)
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
[cloud@202217b2017 Desktop]$ nano socket_server.c
[cloud@202217b2017 Desktop]$ gcc socket_server.c -o server
./server
TCP server socket created.
Bind to the port number: 5566
Listening...

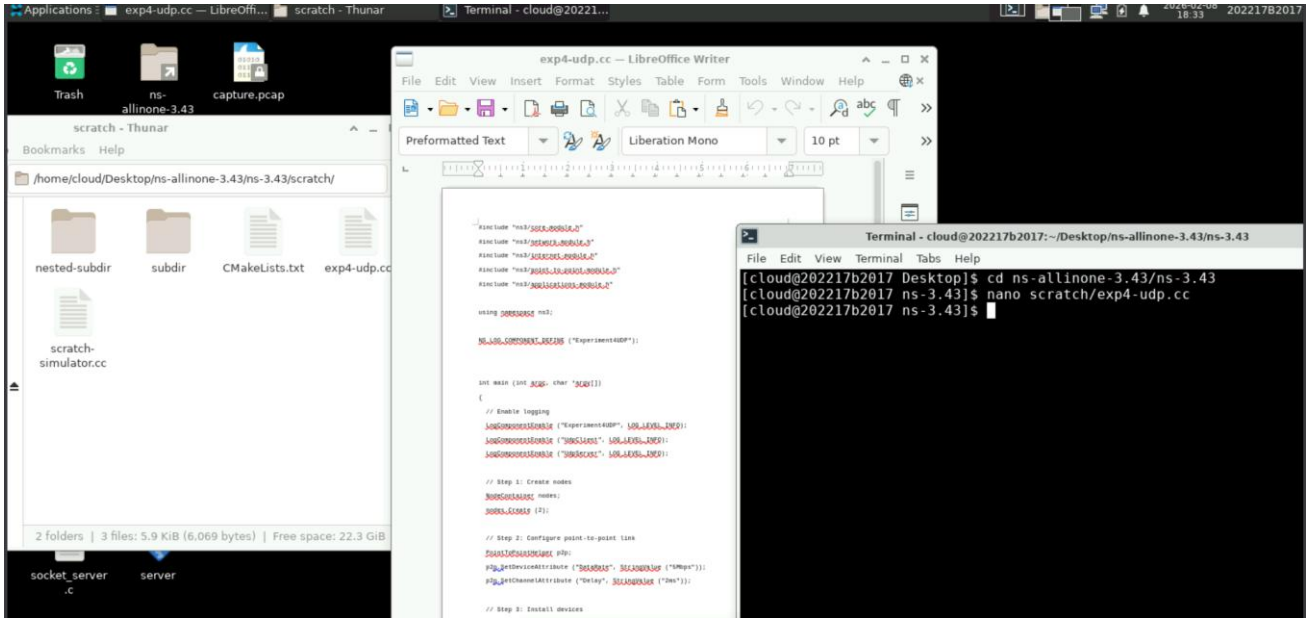

Applications
Xfce Terminal
2026-02-08 18:06
202217B2017
Trash
ns-allinone-3.43
capture.pcap
File System
Wireshark
socket_client.c
Home
Terminal Emulator
LibreOffice 24.8
Firefox
exp1_amithgeojose.pcap...
http capture.pca...
socket_server.c
server
Terminal - cloud@202217b2017:~/Desktop
File Edit View Terminal Tabs Help
[cloud@202217b2017 Desktop]$ nano socket_client.c
[cloud@202217b2017 Desktop]$


Applications
socket_client.c — Libre...
2026-02-08 18:23
202217B2017
socket_client.c — LibreOffice Writer
File Edit View Insert Format Styles Table Form Tools Window Help
Preformatted Text
Liberation Mono
10 pt
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
int main() {
char *ip = "127.0.0.1";
int port = 5566;
int sock;
struct sockaddr_in addr;
socklen_t addr_size;
char buffer[1024];
int n;
sock = socket(AF_INET, SOCK_STREAM, 0);
if (sock < 0) {
perror("Error in Creating.....");
exit(1);
}
printf("TCP server socket created.\n");
memset(&addr, '\0', sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = port;
addr.sin_addr.s_addr = inet_addr(ip);
connect(sock, (struct sockaddr *)&addr, sizeof(addr));
printf("Connected to the server.\n");
bzero(buffer, 1024);
strcpy(buffer, "HELLO, THIS IS CLIENT...What is your BITS ID.");
printf("Client: %s\n", buffer);
send(sock, buffer, strlen(buffer), 0);
bzero(buffer, 1024);
recv(sock, buffer, sizeof(buffer), 0);
printf("Server: %s\n", buffer);
close(sock);
printf("Disconnected from the server.\n");
return 0;
}

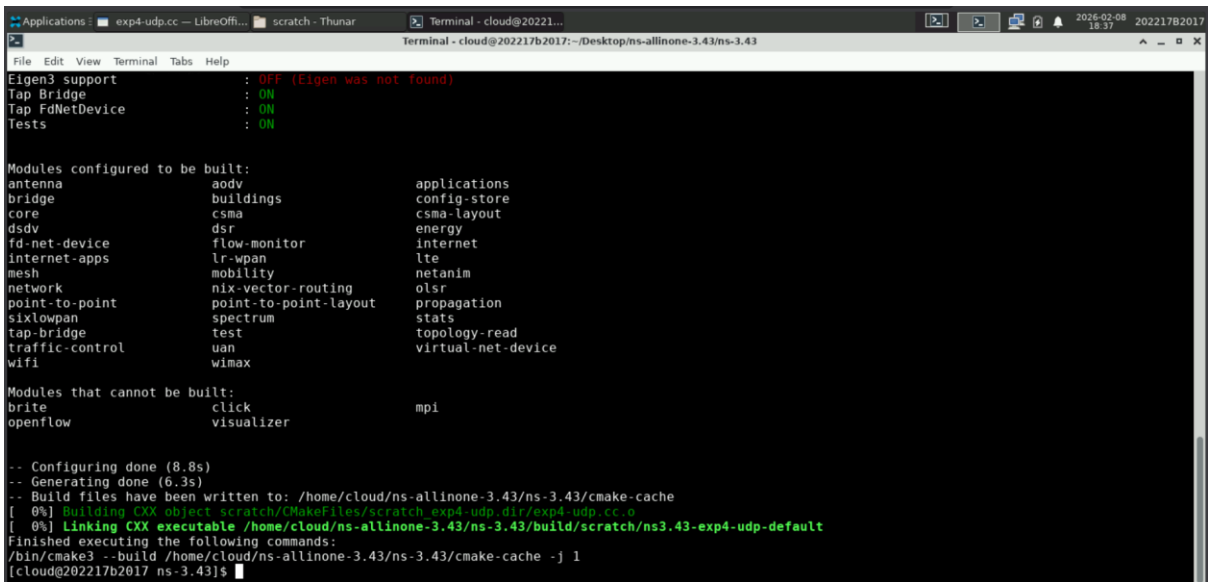


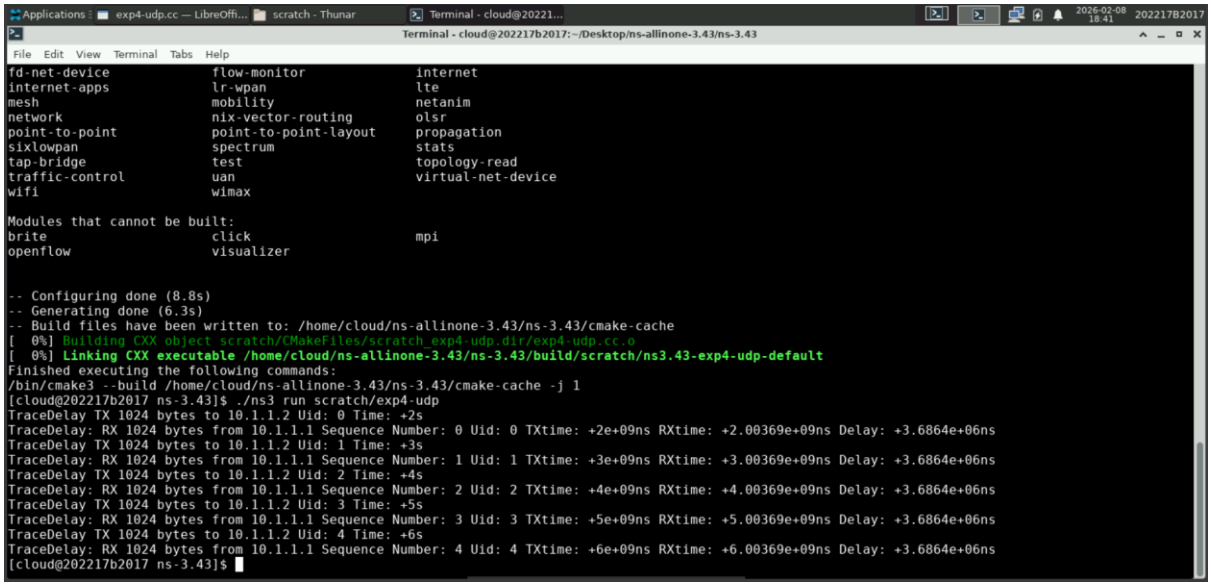
Result: The TCP server successfully accepted a client connection, received a message from the client, and sent a response containing the BITS ID. This confirms successful socket-based communication using TCP.

- Experiment 2: **Building different types of networks for experimenting with different types of applications using Network Simulators(ns-3 Tool)**



Build started:





Observation

The UDP client successfully transmitted five packets of size 1024 bytes to the UDP server at 1-second intervals. The server received each packet, and transmission delay was observed in the terminal output.

Result

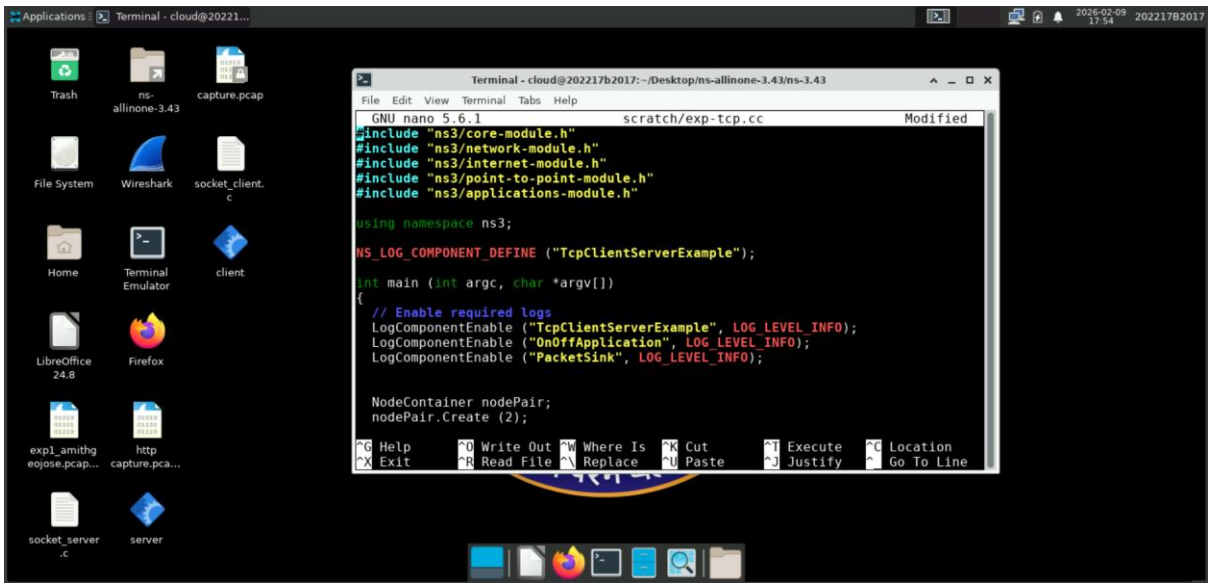
UDP-based client–server communication was successfully simulated using ns-3.

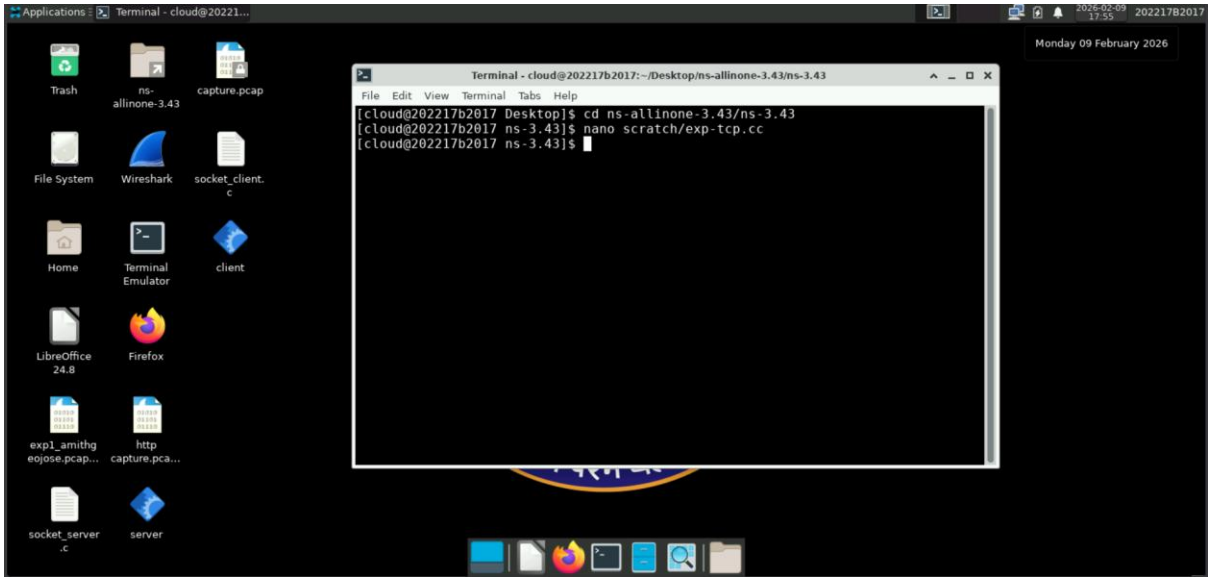
- Experiment 3: **Design a simple network using NS-3 and simulate TCP-based client–server communication to study reliable data transmission**

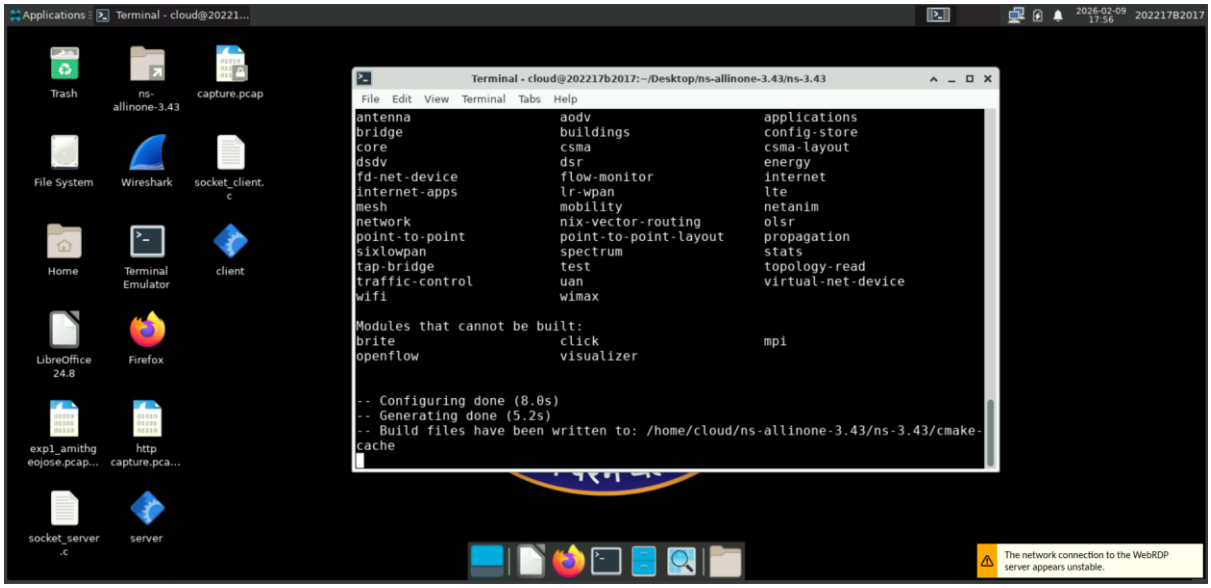
Code:

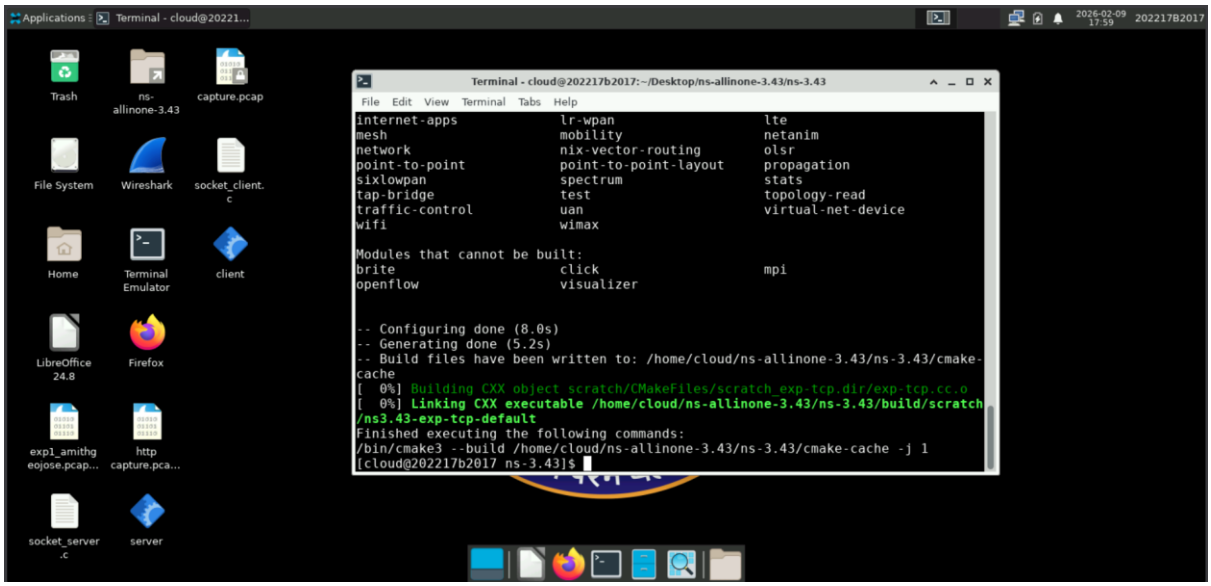
```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("TcpClientServerExample");

int main (int argc, char *argv[])
{
 LogComponentEnable ("TcpClientServerExample", LOG_LEVEL_INFO);
 LogComponentEnable ("OnOffApplication", LOG_LEVEL_INFO);
 LogComponentEnable ("PacketSink", LOG_LEVEL_INFO);


 NodeContainer nodePair;
 nodePair.Create (2);

 PointToPointHelper link;
 link.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
 link.SetChannelAttribute ("Delay", StringValue ("2ms"));

 NetDeviceContainer netDevices = link.Install (nodePair);

 InternetStackHelper internet;
 internet.Install (nodePair);

 Ipv4AddressHelper ipv4;
 ipv4.SetBase ("10.1.3.0", "255.255.255.0");
 Ipv4InterfaceContainer ipInterfaces = ipv4.Assign (netDevices);

 uint16_t serverPort = 8080;
 Address serverAddress (InetSocketAddress (Ipv4Address::GetAny (), serverPort));
 PacketSinkHelper tcpServer ("ns3::TcpSocketFactory", serverAddress);

 ApplicationContainer serverApp = tcpServer.Install (nodePair.Get (1));
 serverApp.Start (Seconds (1.0));
 serverApp.Stop (Seconds (10.0));
```

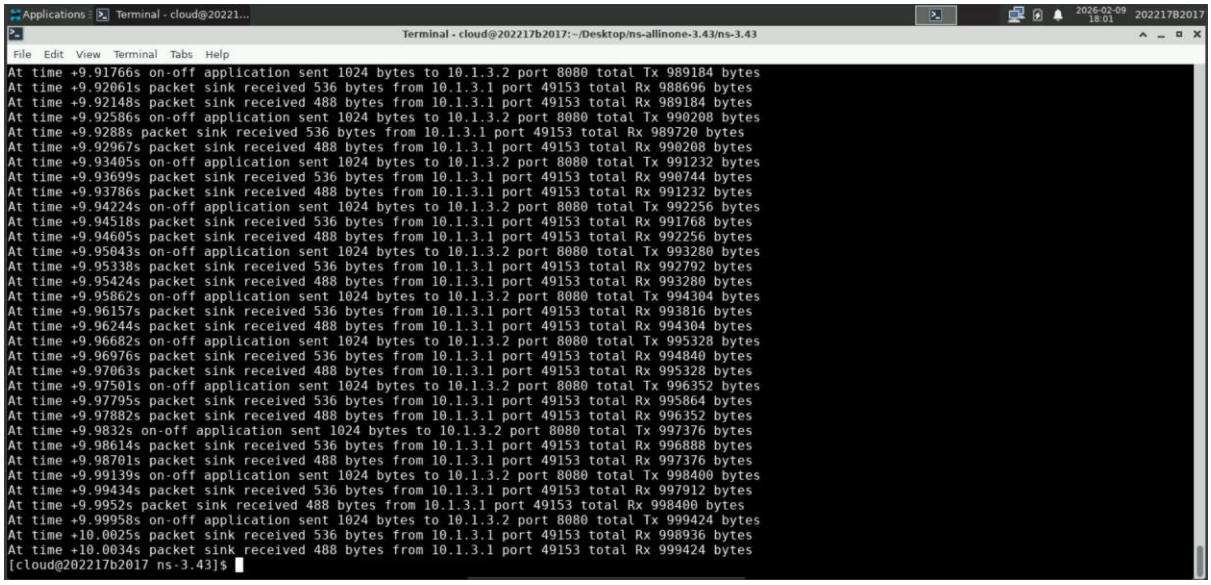
```
Address clientTarget (
  InetSocketAddress (ipInterfaces.GetAddress (1), serverPort));

OnOffHelper tcpClient ("ns3::TcpSocketFactory", clientTarget);
tcpClient.SetAttribute ("PacketSize", UintegerValue (1024));
tcpClient.SetAttribute ("DataRate", StringValue ("1Mbps"));
tcpClient.SetAttribute ("OnTime",
    StringValue ("ns3::ConstantRandomVariable[Constant=1]"));
tcpClient.SetAttribute ("OffTime",
    StringValue ("ns3::ConstantRandomVariable[Constant=0]"));

ApplicationContainer clientApp = tcpClient.Install (nodePair.Get (0));
clientApp.Start (Seconds (2.0));
clientApp.Stop (Seconds (10.0));

Simulator::Run ();
Simulator::Destroy ();

return 0;
}
```




Applications
Terminal - cloud@20221...
2026-02-09 17:55
202217B2017
Monday 09 February 2026
Trash
ns-allinone-3.43
capture.pcap
File System
Wireshark
socket_client.c
Home
Terminal Emulator
client
LibreOffice 24.8
Firefox
exp1_amithgeojose.pcap...
http capture.pca...
socket_server.c
server
Terminal - cloud@202217b2017:~/Desktop/ns-allinone-3.43/ns-3.43
File Edit View Terminal Tabs Help
[cloud@202217b2017 Desktop]$ cd ns-allinone-3.43/ns-3.43
[cloud@202217b2017 ns-3.43]$ nano scratch/exp-tcp.cc
[cloud@202217b2017 ns-3.43]$


Applications
Terminal - cloud@20221...
2026-02-09 17:56
202217B2017
Trash
ns-allinone-3.43
capture.pcap
File System
Wireshark
socket_client.c
Home
Terminal Emulator
client
LibreOffice 24.8
Firefox
exp1_amithgeojose.pcap...
http capture.pca...
socket_server.c
server
Terminal - cloud@202217b2017:~/Desktop/ns-allinone-3.43/ns-3.43
File Edit View Terminal Tabs Help
antenna aodv applications
bridge buildings config-store
core csma csma-layout
dsdv dsr energy
fd-net-device flow-monitor internet
internet-apps lr-wpan lte
mesh mobility netanim
network nix-vector-routing olsr
point-to-point point-to-point-layout propagation
sixlowpan spectrum stats
tap-bridge test topology-read
traffic-control uan virtual-net-device
wifi wimax
Modules that cannot be built:
brite click mpi
openflow visualizer
-- Configuring done (8.0s)
-- Generating done (5.2s)
-- Build files have been written to: /home/cloud/ns-allinone-3.43/ns-3.43/cmake-cache
The network connection to the WebRDP server appears unstable.


Applications
Terminal - cloud@20221...
2026-02-09 17:59
202217B2017
Trash
ns-allinone-3.43
capture.pcap
File System
Wireshark
socket_client.c
Home
Terminal Emulator
client
LibreOffice 24.8
Firefox
exp1_amithgeojose.pcap...
http capture.pca...
socket_server.c
server
Terminal - cloud@202217b2017:~/Desktop/ns-allinone-3.43/ns-3.43
File Edit View Terminal Tabs Help
internet-apps lr-wpan lte
mesh mobility netanim
network nix-vector-routing olsr
point-to-point point-to-point-layout propagation
sixlowpan spectrum stats
tap-bridge test topology-read
traffic-control uan virtual-net-device
wifi wimax
Modules that cannot be built:
brite click mpi
openflow visualizer
-- Configuring done (8.0s)
-- Generating done (5.2s)
-- Build files have been written to: /home/cloud/ns-allinone-3.43/ns-3.43/cmake-cache
[0%] Building CXX object scratch/CMakeFiles/scratch_exp-tcp.dir/exp-tcp.cc.o
[0%] Linking CXX executable /home/cloud/ns-allinone-3.43/ns-3.43/build/scratch/ns3.43-exp-tcp-default
Finished executing the following commands:
/bin/cmake3 --build /home/cloud/ns-allinone-3.43/ns-3.43/cmake-cache -j 1
[cloud@202217b2017 ns-3.43]$



Observation:

A simple two-node network was designed using ns-3 with a point-to-point link. A TCP client running on one node successfully established a connection with a TCP server running on the other node. The client continuously transmitted data packets, and the server reliably received the packets, as observed from the terminal output showing packet transmission, reception, and total bytes transferred.

Conclusion:

TCP-based client–server communication was successfully simulated using ns-3. The experiment demonstrated reliable data transmission, as all transmitted data packets were received correctly by the server, confirming the connection-oriented and reliable nature of the TCP protocol.

Screenshot of lab after completing experiments: